

Audit Technique

RestoProGN — KFM Delice

Application de gestion de restaurant pour KFM Delice, Conakry, Guinee

Date : 9 juin 2026

Version auditee : 1.0 (branche principale)

Stack : Next.js 16.1.3 / Prisma / SQLite / shadcn/ui / JWT

Ce rapport presente un audit complet de l'application RestoProGN : points forts, points faibles, fonctionnalites operationnelles, dysfonctionnements identifies et propositions d'evolution.

Critere	Note	Appreciation
Architecture globale	7/10	Solide mais monolithique
Securite	5/10	Insuffisant pour la production
Experience utilisateur	8/10	Tres bonne, moderne et fluide
Fonctionnalites	8/10	Riche et bien couverte
Qualite du code	6/10	Acceptable mais manque de rigueur TypeScript
Performance	6/10	Polling excessif, pas de WebSocket
Maintenabilite	5/10	Composants monolithiques, pas de tests
SCORE GLOBAL	6.4/10	Prototype avance, pas encore production-ready

Tableau 1 — Synthese des scores par critere

1. Points Forts de l'Application

1.1 Couverture fonctionnelle exceptionnelle

RestoProGN offre un ensemble de fonctionnalités remarquablement complet pour un prototype de gestion restaurant. L'application couvre l'intégralité du cycle de vie d'un établissement de restauration : gestion du menu avec CRUD complet, gestion des commandes (sur place, à emporter, livraison), réservations, suivi des livraisons en temps réel, gestion du personnel, facturation, devis, suivi des dépenses, et même un système de point de vente (POS). Cette couverture fonctionnelle est rare dans un projet de cette envergure et témoigne d'une vision produit ambitieuse et bien pensée.

L'intégration de trois types d'utilisateurs distincts (administrateur, client, livreur) avec des interfaces dédiées et des flux d'authentification séparés est un choix architectural judicieux qui permet à chaque acteur du restaurant d'interagir avec le système selon ses besoins spécifiques. Le compte driver avec GPS tracking en temps réel est particulièrement pertinent pour le contexte guinéen où les livraisons à moto sont prédominantes.

1.2 Experience utilisateur moderne et soignée

L'interface utilisateur tire pleinement parti de shadcn/ui et de Framer Motion pour offrir une expérience fluide et moderne. Le thème sombre/clair avec persistance via next-themes, les animations de scroll avec AnimatedSection, les transitions entre les onglets de l'administration, et les micro-interactions (notifications sonner, compteurs animés, barres de progression) confèrent à l'application un aspect professionnel et agréable à utiliser. La navigation publique avec ses sections (Hero, Menu, Reservation, Avis, À Propos, Footer) suit les meilleures pratiques du web design pour les restaurants.

Le système de cartes SVG pour la carte de Conakry est une solution créative et légère qui évite la dépendance à des services de cartographie externes tout en restant visuellement informative. Les marqueurs de position des livreurs, le rafraîchissement automatique toutes les 15 secondes, et la boussole intégrée ajoutent une couche de professionnalisme appréciable. Le POS avec son système de panier, ses raccourcis de table, et sa génération de ticket de caisse est fonctionnel et bien conçu.

1.3 Adaptation au contexte guinéen

L'application est particulièrement bien adaptée au contexte guinéen. La monnaie est en GNF (Franc Guinéen), les méthodes de paiement incluent Orange Money et MTN Money qui sont les solutions mobile money dominantes en Guinée, les zones de livraison correspondent aux communes réelles de Conakry (Kaloum, Dixinn, Matam, Matoto), et les plats du menu reflètent la cuisine guinéenne et ouest-africaine (Riz Jollof, Plasas, Attieke, Thieboudienne, Mafe, Bissap). Les numéros de téléphone utilisent le format +224, et les images générées pour les plats sont cohérentes avec l'identité visuelle du restaurant. Cette localisation va au-delà du simple traducteur et montre une compréhension profonde du marché cible.

1.4 Stack technique cohérente et moderne

Le choix de Next.js 16 avec l'App Router, Prisma ORM, SQLite, et shadcn/ui constitue une stack technique coherente et productive. Prisma offre un typage fort et des migrations de schema pratiques, SQLite simplifie le deploiement sans serveur de base de donnees externe, et shadcn/ui fournit des composants accessibles et personnalisables. L'utilisation de Zod pour la validation des entrees API, de bcrypt pour le hachage des mots de passe (facteur 12), et de JWT pour l'authentification montre une bonne comprehension des pratiques de securite fondamentales. La validation Zod est systematiquement appliquee sur tous les endpoints API, ce qui est une excellente pratique.

2. Points Faibles et Vulnerabilites

2.1 Securite insuffisante pour la production

Le principal point faible de l'application reside dans sa posture de securite, qui n'est pas adaptee a un deploiement en production. Plusieurs vulnerabilites critiques ont ete identifiees qui pourraient compromettre les donnees des utilisateurs et la integrite du systeme si l'application etait exposee sur Internet sans corrections.

Vulnerabilite	Severite	Impact	Detail
JWT_SECRET code en dur	CRITIQUE	Compromission totale	Le secret JWT est defini en dur dans auth.ts et non dans .env
Absence de rate limiting	ELEVEE	Attaque par force brute	Aucune protection sur les endpoints de login/register
Identifiants demo visibles	MOYENNE	Acces non autorise	mots de passe affiches dans l'interface de login
Pas de refresh token	MOYENNE	Session hijacking	JWT unique 24h sans mecanisme de renouvellement
Pas de middleware Next.js	MOYENNE	Contournement d'auth	Auth verifiee uniquement dans chaque endpoint API
Pas de CORS configure	MOYENNE	Requetes cross-origin	Aucune restriction sur les origines des requetes
Seed sans auth initiale	BASSE	Reinitialisation DB	POST /api/seed accessible sans auth si DB vide

Tableau 2 — Vulnerabilites de securite identifiees

2.2 Architecture monolithique cote client

L'application utilise un pattern de Single Page Application (SPA) complet cote client ou le fichier page.tsx gere l'integralite de la navigation via un etat mode. Ce choix signifie qu'aucune route Next.js n'est utilisee pour les pages publiques ou les tableaux de bord : tout est rendu cote client. Bien que fonctionnel, ce pattern presente plusieurs inconvenients majeurs pour un projet qui vise la production.

Premierement, le SEO est inexistant car les robots des moteurs de recherche ne verront que la page publique initiale sans navigation. Pour un restaurant qui cherche a attirer des clients via le web, c'est

un probleme critique. Deuxiemement, l'absence de routes signifie que les utilisateurs ne peuvent pas partager de liens directs vers des sections specifiques (menu, reservations, avis) ni utiliser les boutons precedent/suivant du navigateur de maniere fiable. Troisiemement, le code de l'application entiere est charge en une seule fois, ce qui degrade les performances de chargement initial a mesure que l'application grandit.

2.3 Composants monolithiques et peu maintenables

Plusieurs composants atteignent des tailles problematiques qui nuisent a la maintenabilite. CustomerAccount.tsx depasse les 800 lignes et gere 7 onglets avec leurs propres formulaires, etats et logiques. AdminDashboard.tsx, bien que plus court (362 lignes), centralise la totalite de l'etat de l'administration (formulaires de chaque onglet, donnees de chaque section) et transmet des props massives aux sous-composants, y compris des valeurs passees comme undefined as any pour contourner les erreurs TypeScript. Ce pattern est fragile et rend le debuggage difficile.

L'absence totale de tests (ni tests unitaires, ni tests d'integration, ni tests end-to-end) aggrave ce probleme. Chaque modification du code peut introduire des regressions sans qu'aucun filet de securite automatise ne puisse les detecter. Pour un projet de cette taille avec autant de fonctionnalites interconnectees, l'absence de tests est un risque majeur pour la stabilite a long terme.

2.4 TypeScript configure de maniere permissive

La configuration TypeScript est volontairement permissive : ignoreBuildErrors est active dans next.config.ts, et noImplicitAny est desactive dans tsconfig.json. Ces parametres permettent au developpeur de contourner les erreurs de typage au lieu de les corriger, ce qui conduit a un code moins sur et plus difficile a refactorer. L'utilisation recurrente de undefined as any dans AdminDashboard.tsx pour passer des props manquantes est un symptome direct de ce probleme. Ces contournements masquent des bugs potentiels et rendent le code plus fragile.

2.5 Polling excessif au lieu de WebSocket

L'application utilise le polling HTTP pour les fonctionnalites en temps reel, avec trois intervalles distincts : les notifications admin (30 secondes), la carte des livreurs (15 secondes), et le suivi de commande client (5 secondes). Ce pattern est inefficace a plusieurs niveaux. Chaque requete de polling consomme des ressources serveur meme quand il n'y a pas de nouvelles donnees, le delai entre les mises a jour peut etre perceptible pour les utilisateurs (5 secondes pour le suivi client), et la charge reseau augmente lineairement avec le nombre d'utilisateurs connectes. Le projet contient deja des exemples WebSocket dans le repertoire examples/ mais ils ne sont pas integres.

3. Fonctionnalites Operationnelles

L'audit a verifie que les fonctionnalites suivantes sont operationnelles dans le code et fonctionnent correctement dans le flux nominal. Chaque module a ete analyse du point de vue du code source, des endpoints API, et de la coherence des donnees.

Module	Fonctionnalite	Statut
Site public	Page d'accueil avec Hero, Menu, Reservation, Avis, A Propos, Footer	Operationnel
Site public	Navigation responsive avec menu hamburger mobile	Operationnel
Site public	Bouton WhatsApp flottant pour commandes	Operationnel
Authentification	Login Admin (3 roles : admin, manager, staff)	Operationnel
Authentification	Login/Inscription Client avec validation	Operationnel
Authentification	Login Driver avec JWT dedie	Operationnel
Administration	13 onglets : Vue d'ensemble, Commandes, Menu, Livraisons, Livreurs, Reservations, Avis, Personnel, Admins, Factures, Devis, Depenses, POS	Operationnel
Menu	CRUD complet avec 5 categories (21 plats), filtres, badges	Operationnel
Commandes	Cycle complet : pending, preparing, ready, delivering, delivered	Operationnel
Livraisons	Carte SVG Conakry, position livreurs temps reel, assignation	Operationnel
POS	Panier, choix table, modes paiement, ticket de caisse	Operationnel
Compte client	7 onglets : dashboard, commandes, reservations, avis, fidelite, commande en ligne, profil	Operationnel
Compte livreur	4 onglets : commandes, carte, historique, profil avec GPS	Operationnel
Fidelite	Systeme de points avec paliers Bronze/Argent/Or/Platine	Operationnel
Theme	Mode clair/sombre avec persistance localStorage	Operationnel
Base de donnees	Seeding automatique avec donnees realistes guineennes	Operationnel

Tableau 3 — Inventaire des fonctionnalites operationnelles

4. Dysfonctionnements et Problemes Identifies

4.1 Problemes fonctionnels

4.1.1 Les images du POS ne sont pas toutes coherentes

Bien que le code de PosTab.tsx affiche correctement les images des plats lorsque le champ image est renseigne dans la base de donnees (avec fallback sur des icones de categorie), certaines incoherences subsistent. Dans les donnees de seed, le Plateau Fruits de Mer et les Crevettes Sauce Curry utilisent tous les deux /images/kfm-dish-2.png, et les desserts (Assiette de Fruits Tropicaux et Gateau Chocolat-Coco) partagent /images/kfm-dish-3.png. Cela signifie que des plats differents affichent la meme image dans le POS et dans le menu public, ce qui peut confondre le personnel et les clients. Chaque plat devrait avoir une image unique et representative.

4.1.2 Pas de page publique accessible sans login

Bien que les composants de la page publique existent (HeroSection, MenuSection, ReservationSection, AvisSection, AboutSection, PublicFooter), la page n'est pas accessible via une URL dediee. Tout est gere par le mode state dans page.tsx. Il n'existe pas de route Next.js comme /menu ou /public que l'on pourrait partager directement. Pour un restaurant, pouvoir partager un lien vers le menu (par exemple sur les reseaux sociaux ou via WhatsApp) est essentiel pour attirer de nouveaux clients.

4.1.3 Annee de copyright obsolete

Le composant PublicFooter affiche un copyright avec 2024 au lieu de 2026. C'est un detail mineur mais qui donne une impression de negligence aux visiteurs du site. L'annee devrait etre dynamique (new Date().getFullYear()) pour eviter ce probleme a l'avenir.

4.2 Problemes techniques

Probleme	Categorie	Description
undefined as any	TypeScript	AdminDashboard passe des props undefined castees en any aux sous-composants (formatPrice, statusColors, etc.)
Pas d'upload d'images	Fonctionnalite	Les images du menu sont des chemins codes en dur dans le seed ; aucun formulaire d'upload dans l'admin
Pas de pagination API	Performance	Les endpoints GET retournent toutes les donnees sans pagination, ce qui ne scale pas
Pas de migrations	Base de donnees	Utilisation de prisma db push au lieu de migrations proper ; pas de historique des changements de schema
Pas de tests	Qualite	Zero fichier de test dans le projet ; aucune verification automatique de regression
Mot de passe driver vide par default	Securite	Le schema Driver a password @default('') ; un livreur cree via l'admin ne peut pas se connecter
Pas de soft delete	Donnees	Les suppressions sont definitives (onDelete: Cascade) ; impossible de recuperer des donnees effacees
Pas de validation commande	Business logic	Le total de la commande est envoye par le client sans verification serveur contre les prix du menu

Tableau 4 — Problemes techniques identifies

4.3 Problemes de performance

L'application presente plusieurs problemes de performance qui pourraient affecter l'experience utilisateur a mesure que le volume de donnees augmente. Le seeding de la base de donnees est declenche automatiquement a chaque chargement de page via un useEffect dans page.tsx, ce qui signifie que chaque visiteur declenche un appel POST /api/seed (bien que celui-ci soit protege par une verification d'existence des donnees, il represente un appel reseau inutile a chaque navigation).

Les endpoints API ne disposent d'aucun mecanisme de pagination : tous les enregistrements sont retournes en une seule requete. Pour les commandes, les reservations et les elements du menu, cela

fonctionne avec le volume actuel de données de demo, mais deviendra problématique avec des centaines ou milliers d'enregistrements. Le polling à intervalles réguliers (5s, 15s, 30s) génère une charge réseau constante qui ne scale pas bien avec le nombre d'utilisateurs simultanés.

5. Propositions d'Evolution et d'Amélioration

Les propositions ci-dessous sont classées par priorité et organisées en trois horizons temporels : court terme (1-2 semaines), moyen terme (1-2 mois), et long terme (3-6 mois). Chaque proposition inclut une estimation d'effort et un impact attendu.

5.1 Priorité 1 — Sécurité (Court Terme, 1-2 semaines)

P1.1 — Déplacer JWT_SECRET dans les variables d'environnement

Le secret JWT doit impérativement être stocké dans le fichier .env et jamais codé en dur dans le source. Il s'agit d'une correction critique qui prend moins de 30 minutes mais qui élimine la vulnérabilité la plus grave de l'application. Ajouter JWT_SECRET au fichier .env, modifier auth.ts pour supprimer la valeur par défaut, et s'assurer que le serveur refuse de démarrer si la variable est absente.

P1.2 — Implémenter le rate limiting sur les endpoints d'authentification

Ajouter un middleware de rate limiting sur /api/login, /api/customer-login, /api/customer-register, et /api/driver-login pour limiter les tentatives de connexion à 5 par minute par adresse IP. Cela peut être implémenté avec un simple compteur en mémoire ou via un package npm comme rate-limiter-flexible. Cette protection est essentielle pour empêcher les attaques par force brute sur les mots de passe.

P1.3 — Implémenter un middleware Next.js pour la protection des routes API

Créer un fichier middleware.ts à la racine de src/ qui vérifie le JWT sur toutes les routes API protégées avant qu'elles n'atteignent les handlers. Cela centralise la logique d'authentification, réduit la duplication de code dans chaque endpoint, et empêche les oublis accidentels de vérification d'authentification.

P1.4 — Masquer les identifiants demo de l'interface

Supprimer l'affichage des identifiants de démonstration (admin@kfm-delice.com / kfm2024, etc.) des formulaires de login. Ces informations ne devraient être accessibles que dans la documentation développeur, pas dans l'interface utilisateur. Remplacer par un lien vers la documentation ou les afficher uniquement en mode développement.

5.2 Priorité 2 — Architecture et Qualité (Moyen Terme, 1-2 mois)

P2.1 — Migrer vers le routing Next.js App Router

Remplacer le système de navigation par état (useState mode) par de vraies routes Next.js. La structure proposée serait : / (page publique), /menu (menu public), /reservation (reservation publique), /admin (dashboard admin), /admin/orders, /admin/menu, /client (compte client), /driver (compte livreur), /tracking/[orderId] (suivi de livraison). Cette migration apporte le SEO, le partage de liens, le

chargement par route (code splitting automatique), et la possibilité d'utiliser les Server Components de Next.js pour réduire le JavaScript envoyé au client.

P2.2 — Décomposer les composants monolithiques

Découper `CustomerAccount.tsx` (800+ lignes) en sous-composants indépendants (`ClientDashboardTab`, `ClientOrdersTab`, `ClientReservationsTab`, etc.) chacun dans son propre fichier. Faire de même pour `AdminDashboard.tsx` en rendant les onglets autonomes avec leurs propres états locaux plutôt que de tout centraliser dans le parent. Chaque onglet devrait importer directement les utilitaires de `constants.ts` plutôt que de les recevoir via des props passées en `undefined` ou `any`.

P2.3 — Activer le typage strict TypeScript

Désactiver `ignoreBuildErrors` dans `next.config.ts`, activer `noImplicitAny` et `strict` dans `tsconfig.json`, et corriger toutes les erreurs résultantes. Cela peut sembler fastidieux mais c'est un investissement qui paie considérablement en termes de prévention de bugs et de facilité de refactoring. L'élimination des `undefined` ou `any` est la première priorité.

P2.4 — Ajouter une suite de tests

Mettre en place une suite de tests avec Vitest (tests unitaires), Testing Library (tests de composants React), et Playwright (tests end-to-end). Commencer par les parties les plus critiques : les endpoints d'authentification, le flux de commande, et le système de paiement. Viser un minimum de 60% de couverture sur les endpoints API et les composants principaux. Les tests sont aussi importants que le code lui-même pour garantir la fiabilité à long terme.

P2.5 — Implémenter l'upload d'images pour le menu

Ajouter un système d'upload d'images côté admin qui permet de télécharger des photos de plats directement depuis l'interface. Utiliser le système de fichiers public de Next.js ou un service de stockage comme Cloudinary. L'endpoint `/api/menu` doit accepter des fichiers `multipart/form-data`, et le formulaire `MenuTab` doit inclure un composant d'upload avec aperçu. Cela rend l'application autonome pour la gestion du menu sans dépendre de modifications manuelles du code source.

5.3 Priorité 3 — Performance et Temps Reel (Moyen Terme, 1-2 mois)

P3.1 — Remplacer le polling par des WebSockets

Intégrer Socket.io ou le WebSocket natif de Next.js pour les fonctionnalités temps réel : notifications admin, position des livreurs sur la carte, suivi de commande client, et mises à jour de statut. Les exemples WebSocket existent déjà dans le répertoire `examples/` et servent de base de travail. Les WebSockets éliminent le délai de polling, réduisent la charge serveur de 90%, et offrent une expérience utilisateur significativement plus fluide et réactive.

P3.2 — Ajouter la pagination aux endpoints API

Implémenter la pagination sur tous les endpoints GET qui retournent des listes (orders, reservations, menu, reviews, drivers, etc.) en utilisant les paramètres `page` et `limit`. Retourner le nombre total d'enregistrements dans la réponse pour que le front-end puisse afficher la pagination correctement. Ajouter le tri et le filtrage côté serveur pour éviter de charger toutes les données en mémoire.

P3.3 — Optimiser le seeding automatique

Supprimer l'appel systematique a POST /api/seed dans le useEffect de page.tsx. Le seeding ne devrait etre effectue qu'une seule fois, lors de la premiere initialisation de l'application. Verifier cote serveur si les donnees existent deja avant de les creer, et ne plus appeler cet endpoint a chaque chargement de page. Cette optimisation elimine un appel reseau inutile a chaque visite.

5.4 Priorite 4 — Fonctionnalites Metier (Long Terme, 3-6 mois)

P4.1 — Integration paiement Orange Money / MTN Money

Implementer les API de paiement Orange Money et MTN Money pour permettre aux clients de payer en ligne directement depuis l'application. Actuellement, les methodes de paiement sont enregistrees mais aucun paiement reel n'est effectue. L'integration des API de mobile money est un differentiateur competitif majeur en Guinee ou ces services sont predominants. Cela inclut la generation de liens de paiement, la gestion des callbacks de confirmation, et le suivi des transactions.

P4.2 — Notifications push et SMS

Ajouter un systeme de notifications push (via les Web Push API et Service Workers) et SMS (via une passerelle comme Twilio ou une solution locale guineenne) pour informer les clients du statut de leur commande, les livreurs des nouvelles missions, et les administrateurs des evenements importants. Les notifications sont essentielles pour la retention et la reactivite dans le contexte de la livraison de repas.

P4.3 — Dashboard analytique avance

Enrichir l'onglet Vue d'ensemble avec des graphiques d'evolution du chiffre d'affaires, des plats les plus vendus, des pics de commande par heure et jour, du temps moyen de livraison, et de la satisfaction client. Ces metriques sont indispensables pour la prise de decision business. Utiliser des librairies comme Recharts ou Chart.js pour des visualisations interactives et informatives.

P4.4 — Application mobile (PWA ou native)

Convertir l'application en Progressive Web App (PWA) avec un manifest.json, un Service Worker pour le fonctionnement hors ligne, et l'installation sur l'ecran d'accueil. Cela permet aux livreurs d'utiliser l'application comme une app native sur leur telephone, avec acces au GPS et notifications push. Alternativement, developper une application native avec React Native pour une meilleure integration avec les fonctionnalites du telephone (GPS en arriere-plan, notifications, etc.).

P4.5 — Verification du prix des commandes cote serveur

Implementer une validation cote serveur qui recalcule le total de la commande a partir des prix actuels des plats en base de donnees, plutot que de faire confiance au total envoye par le client. Cela empeche la manipulation des prix et garantit l'integrite financiere des transactions. Ajouter egalement une verification de la disponibilite des plats et des heures d'ouverture.

6. Feuille de Route Recommandee

Phase	Duree	Actions cles	Livrables
Phase 1 : Securite	1-2 sem.	JWT_SECRET dans .env, rate limiting, middleware Next.js, masquer identifiants demo	Application securisee pour pre-production
Phase 2 : Architecture	3-4 sem.	Migration vers App Router, decomposition des composants, TypeScript strict, suite de tests	Code base maintainable et type-safe
Phase 3 : Performance	2-3 sem.	WebSocket temps reel, pagination API, optimisation seeding, upload d'images	Application performante et scalable
Phase 4 : Business	6-12 sem.	Paieement Orange/MTN Money, notifications push/SMS, dashboard analytique, PWA	Application production-ready avec revenus

Tableau 5 — Feuille de route recommandee par phases

Ce rapport d'audit a ete realise sur la base d'une analyse approfondie du code source de RestoProGN (KFM Delice) en date du 9 juin 2026. Les recommandations sont formulees dans le but de faire evoluer l'application d'un prototype avance vers un produit deployable en production de maniere fiable et securisee.